

CMPT 353 - Computational Data Science

Final Project: *Sensors, Noise, and Walking*

Zhiyu Mao: 301563547

Ke Wang: 301541496

Brandon Yang: 301558726

Overview:

This project aims to leverage smartphone accelerometer data to classify different modes of human locomotion, such as walking, running, climbing stairs. By analyzing the unique acceleration patterns generated by each activity, we seek to develop a machine learning model capable of automatically identifying the user's movement type in real time.

Data Collection:

We collected data for four different motion modes: running, walking, ascending stairs, and descending stairs. In order to control possible variables that could influence the result, we chose two fixed routes for data collection. For running and walking, we selected track filed in SFU athletic field track, half a circle (around 200m) is taken as a piece of data. For ascending and descending stairs, we selected the stairs from P2 to P7 in SFU Bennett Library as a piece of data.

As for where to place the sensor (smartphone) on the person can collect the best data, we chose three placements: the hand, the pocket, and the ankle.

Therefore, we have 3 different placements, 4 different movement patterns, a total of 12 combinations. For each combination, we collected 3 sets of data. Finally, we collected a total of 36 sets of csv data to use as raw data.

Data Analysis Problems:

1. What are the best features for distinguishing movement types?
2. Which filter is the best for the data?
3. Where is the best placement of sensor? Hand, pocket or ankle?
4. What machine learning model performs the best in classifying each motion mode?

Question 1: What are the best features for distinguishing movement types?

The CSV file we measured contains the following columns: time, gFx, gFy, gFz, and TgF. Initially, we used gFz alone as the vertical acceleration component, and computed horizontal motion using $\sqrt{gFx^2 + gFy^2}$, which we refer to as the horizontal magnitude. While these features showed some periodicity corresponding to step patterns, they were highly sensitive to device orientation and placement. A phone held in the hand, for example, may rotate slightly with each step, causing the vertical and horizontal axes to shift, and distorting the feature signal.

To address this issue, we use the total acceleration magnitude, computed as:

$$\text{acc_total} = \sqrt{gFx^2 + gFy^2 + gFz^2}$$

Although we also considered using TgF (the sensor's force reading) as a feature, we found that it lacked rhythmic structure and was sensitive to pressure, making it unreliable for detecting consistent movement patterns. In contrast, the total acceleration feature consistently highlights the cyclic nature of steps and movement bursts, offering both clarity and robustness. For these reasons, total acceleration is the most effective and interpretable feature for our gait and activity analysis.

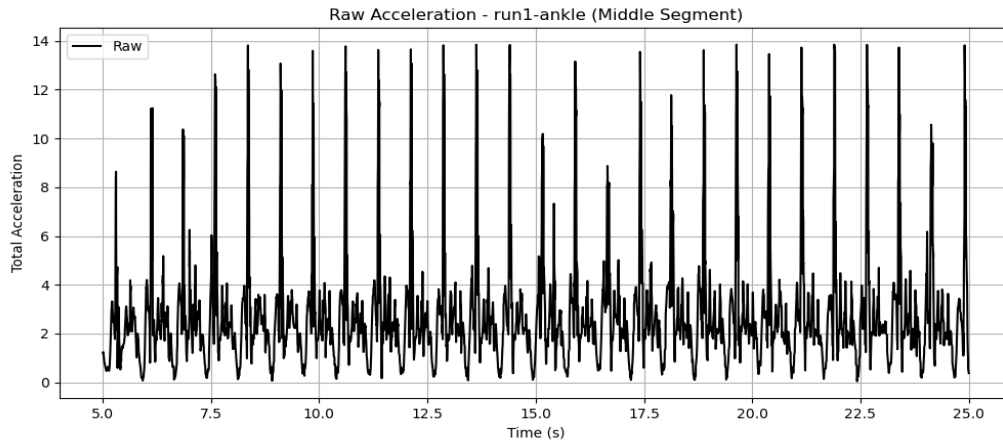
Question 2: Which filter is the best for the data?

As concluded in Question 1, we selected acc_total, the total acceleration magnitude as the raw data, we now focus on how best to filter this raw signal to reduce sensor noise while preserving meaningful motion patterns.

Since the first few seconds of data include the transition from when the “Start Recording” button is clicked to when actual movement begins, we exclude this initialization period. Specifically, we extract the data from 5 seconds to 25 seconds, which contains clean, meaningful motion information suitable for analysis and filtering.

Out of all 12 movement-placement combinations we collected, we selected the "run + ankle" combination for detailed analysis. This combination contains the most noticeable noise, likely due to sensor vibrations and impact shock from ground contact during running. In the raw acceleration plot, we observe large peaks, which correspond to individual steps—these are features we want to preserve. However, the high-frequency

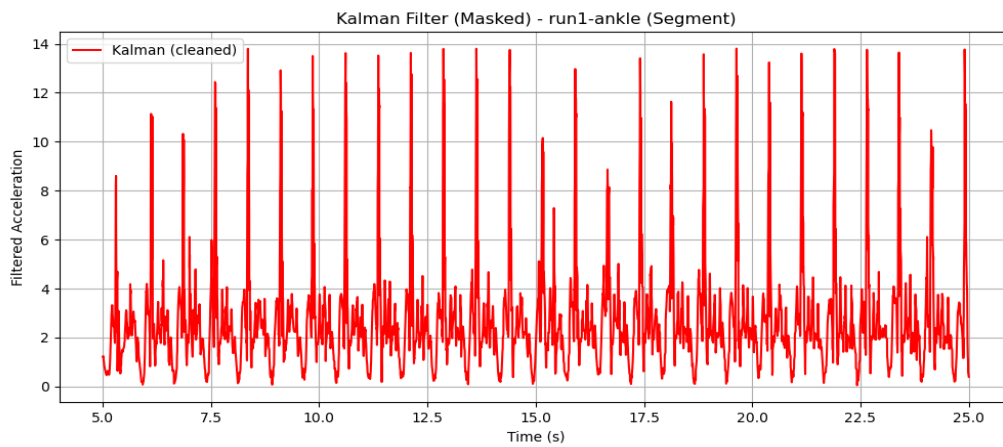
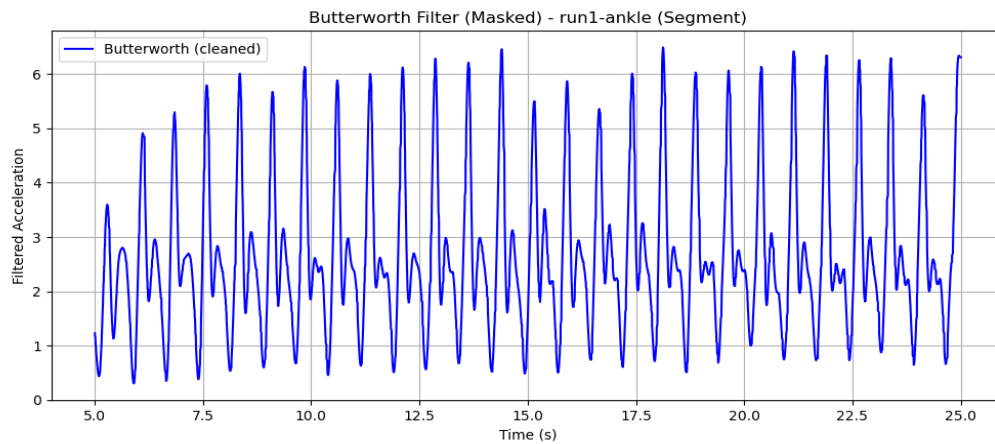
jitter and dense, rapidly varying segments between the peaks represent noise introduced by the sensor and must be filtered out.



To process the signal, we apply three types of filters:

- **Butterworth low-pass filter**, which cleanly removes high-frequency noise while preserving the step signal's shape and frequency content.
- **Loess filter**, a local regression smoother that works well on gentle trends but may oversmooth sharp features like step peaks.
- **Kalman filter**, which adapts to changing signals but depends on correctly chosen process and measurement noise models.

To further enhance signal clarity, we implement a post-filter masking step called `mask_low_variation`. This function calculates the gradient (first derivative) of the filtered signal and masks out regions where the acceleration varies very little—typically flat areas or jittery segments that don't contribute to meaningful step cycles. This helps ensure that only segments with significant, dynamic motion are visualized and analyzed, reducing distraction from noisy or stationary segments.



Comparing three filtered results we get, we find that the Butterworth filter, when combined with the mask_low_variation post-processing, delivers the cleanest and most interpretable

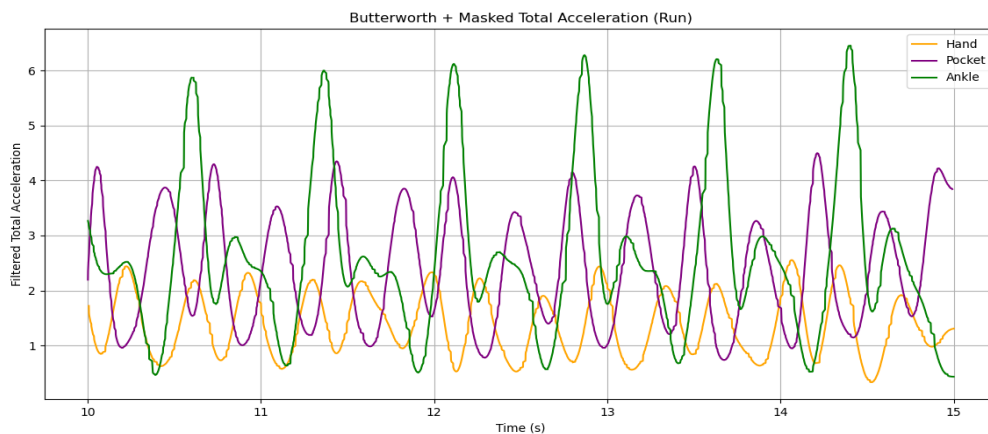
signal. While all three filters—Butterworth, Loess, and Kalman—help to reduce some level of noise, Butterworth stands out in its ability to effectively suppress high-frequency jitter, while the other two filters still leave a lot of noise unfiltered.

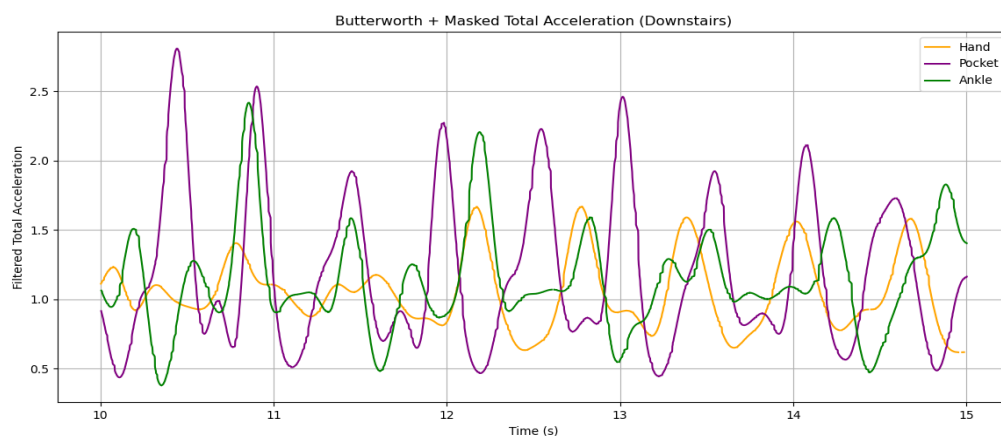
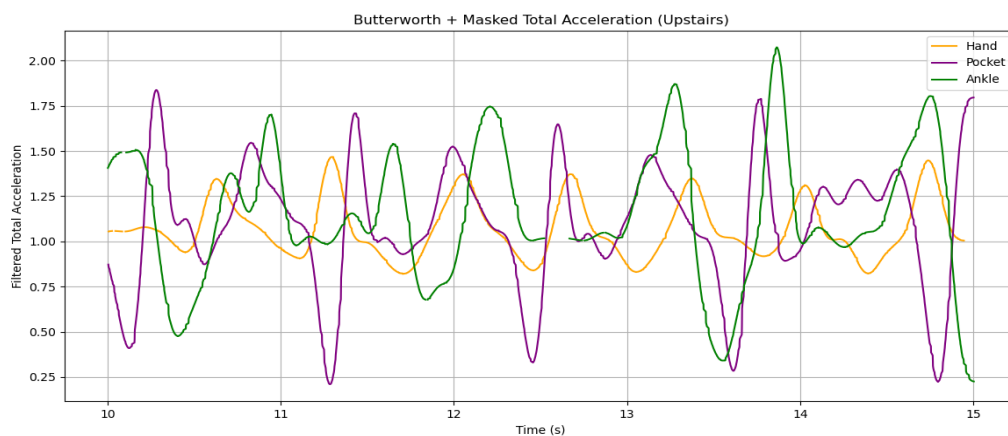
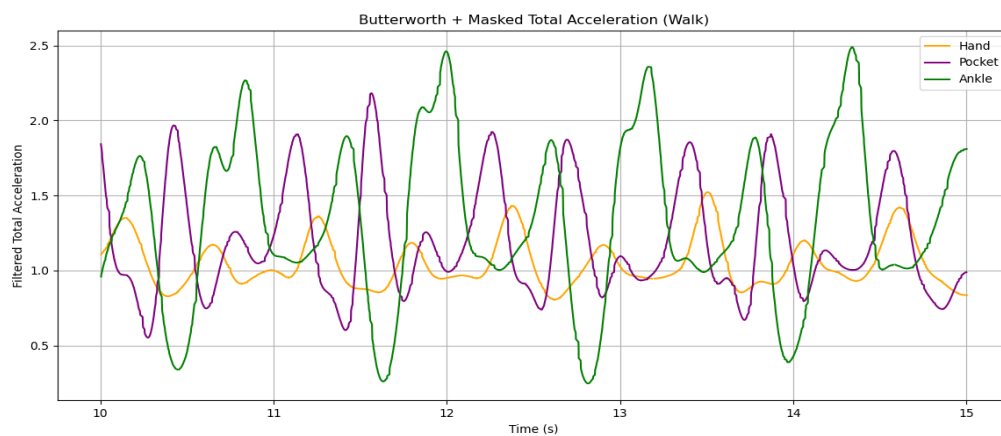
Question 3: Where is the best placement of sensor? Hand, pocket or ankle?

When the goal is to capture clean, informative features from human motion—especially for detecting steps, gait rhythm, or running cycles—the placement of the sensor is critical.

We have decided to use total acceleration for raw data and filtered with butterworth and masked filter, now we compare how the sensor performs in different placements of body. We extract the data from 10 second to 15 second to avoid possible beginning noises and better visualization of data feature.

We plotted the filtered and masked total acceleration signals for each sensor placement on the same graph for each movement type. This allowed direct visual comparison of peak structure (sharpness, periodicity) and signal smoothness.





For running, the ankle consistently showed the strongest, cleanest peaks aligned with step timing. For walking and stair climbing, the ankle still performed reliably, showing good structure with minimal noise. In the downstairs case, the pocket showed higher peak amplitudes, but further analysis revealed this was likely caused by extra movement of the phone inside loose clothing rather than true leg motion. The ankle provided more stable, biomechanically aligned data.

Across all motion types, the ankle placement produced the clearest, most structured step patterns, with sharp, periodic peaks that reliably correspond to individual steps. While other placements occasionally showed high peaks, they were more affected by random motion artifacts. Therefore, for reliable feature extraction and consistent gait analysis, the ankle is the best placement for the smartphone sensor.

Question 4: What machine learning model performs the best in classifying each motion mode?

With previous work we have decided to use the ankle data for further training. As mentioned in the Limitation section, we found that the models had difficulty distinguishing between walking upstairs and downstairs. To address this, we collected several additional data samples but ultimately decided to treat both as a single "stairs" category.

To better evaluate and compare the performance of different machine learning methods, we chose to feed each model with all the available data we had collected.

To evaluate which machine learning model performs best at classifying motion types, we first implemented a data processing pipeline using Python. We began by loading raw accelerometer data from ankle-mounted sensors and then applied a Butterworth low-pass filter to reduce noise. Next, we used a gradient-based method to mask out low-variation regions in the signal. From each recording, we computed four key statistical features: mean, standard deviation, range, and median of the filtered signal. Since the models had difficulty distinguishing between "upstair" and "downstair" movements, we merged both into a single "stairs" category to improve classification consistency.

Below are the results:

- **Random Forest:**

An ensemble-based method that combines multiple decision trees to improve generalization.

Result: Mean Accuracy = 0.96, Standard Deviation = ± 0.08

- **K-Nearest Neighbors (KNN):**

An instance-based learning method that classifies a point based on the majority class of its nearest neighbors.

Result: Mean Accuracy = 0.76, Standard Deviation = ± 0.15

- **Naive Bayes:**

A probabilistic model that assumes feature independence to compute class likelihoods.

Result: Mean Accuracy = 0.84, Standard Deviation = ± 0.15

The results show that Random Forest achieved the highest accuracy and consistency, making it the most effective model for this task.

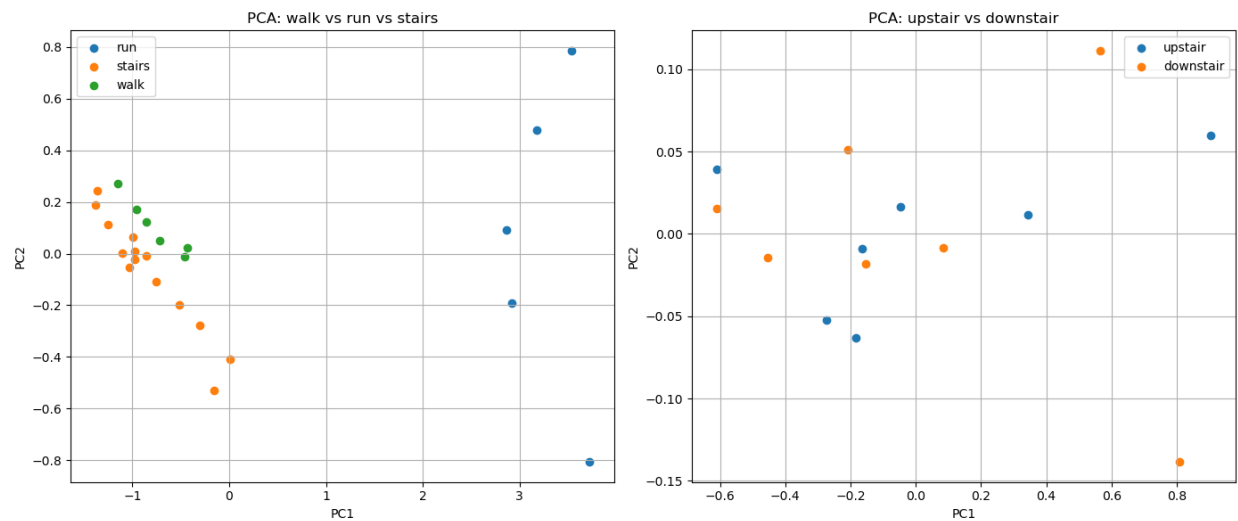
Web Application:

A website was developed to analyze collected data, which can be accessed by running `app.py` to open a local port. The website provides two main functions. The first is "Train Data", used for training models. Several training methods are implemented, including Random Forest, K-Nearest Neighbors (KNN), and Naive Bayes. In this function, multiple CSV files containing data can be uploaded, and the data type for each file must be specified. After uploading all the data, clicking the "Train Model" button initiates model training and returns the prediction accuracy. Before feature extraction, a Butterworth filter is applied to smooth the data and reduce noise, resulting in better model performance. The trained models are saved in the "models" folder, and the CSV files used for training are stored in the "training_data" folder. The second function, "Analyze Data," allows testing of the trained model. An average step length can also be entered to estimate speed. After uploading the data and clicking the "Analyze Data" button, the website displays gait analysis results and visualizations generated by the model, enabling verification of whether the model was trained correctly.

Limitation:

Although we tried many ways to distinguish the two motion modes of going up and down stairs, the results of machine learning were not satisfactory, and they could only be classified as "stairs". In order to better distinguish them, we need more sets of data for training, and the amount of data we have collected is not enough.

According to the PCA graphs below, the left plot shows that run, walk, and stairs are generally well separated, indicating that the features are effective for classifying these motions. However, the right plot reveals that upstairs and downstairs have significant overlap in the PCA space, making them difficult to distinguish. This supports our decision to merge them into a single stairs category.



Experience Summary:

Zhiyu Mao:

- Brainstormed with group members to come up with the project design
- Collecting data used for training and testing
- Optimizing data filtering for better fit in model training
- Implementing the training code based on the web we designed

Ke Wang:

- Brainstormed with group members to come up with the project design
- Collecting data used for training and testing
- Designed web to handle training method
- Created Jupyter Notebook support for question 4
- Created Jupyter Notebook support for proving limitation
- Contributed to writing the report

Brandon Yang:

- Brainstormed with group members to come up with the project design
- Listed key questions related to data analysis
- Created Jupyter Notebook support for questions 2 and 3
- Contributed to writing the report
- Managed the Git repository and version control